Combinar consultas, subconsultas y CTE

`UNION`, `INTERSECT`, `EXCEPT`, `EXISTS` y consultas con nombre

Semana 12 - Sesión 24

Rel Guzman



Logro de la sesión

Al finalizar, podrás **combinar** los resultados de dos consultas, escribir **subconsultas** con `IN` y `EXISTS`, y ordenar una consulta larga por pasos con **CTE**, en MySQL y SQL Server.

Revisión de la sesión anterior: conectarse a un gestor

Para conectarte usas los **datos de conexión**:

Dato	En clase
Host	`localhost`
Puerto	`3306`
Usuario	`root`
Contraseña	la de la instalación

También puedes iniciar la conexión hacia una **base de datos**, como `peluqueria`.

Si no conecta, revisa en este orden:

1. ¿El **servicio** está encendido?
2. ¿Los **datos de conexión** son correctos?

Hoy

Ya sabes conectarte. Hoy creamos `peluqueria` en los dos gestores y le hacemos preguntas más difíciles.

Práctica 1 Crear la base en los dos gestores

Los scripts del modelo físico están en `ejemplos/practicas/`.

1. **MySQL:** en Workbench, abre y ejecuta `practica_1_modelo_fisico_mysql.sql`.
2. **SQL Server:** en SSMS, abre y ejecuta `practica_1_modelo_fisico_sqlserver.sql`.

Comprueba en cada gestor que la base se creó:

```
USE peluqueria;
SELECT nombre FROM empleado;
```

Ojo con esto

Para repetir en MySQL los ejemplos de clase, ejecuta también `ejemplos/consultas_peluqueria/1_datos_de_ejemplo.sql`. Este script **borra** los clientes, mascotas, citas y pagos, y los vuelve a cargar.

Combinar SELECT

Una pregunta real: ¿quién tiene descuento?

¿Qué empleados también son clientes?

Empecemos con lo que ya sabes: una consulta por tabla.

Clientes

```
SELECT dni, nombre FROM cliente;
```

resultado

dni	nombre
45123678	Lucía Ramos
46234789	Pedro Salas
47345890	Rosa Medina
48456901	Bruno Díaz

Empleados

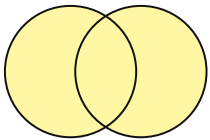
```
SELECT dni, nombre FROM empleado;
```

resultado

dni	nombre
43567012	Ana Torres
48456901	Bruno Díaz
44678123	Carla Núñez

Dos listas, tres preguntas

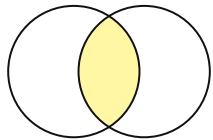
Con esas dos listas puedes responder tres preguntas distintas, y cada una lleva a una decisión.



UNION

¿Quién está en **alguna** de las dos listas?

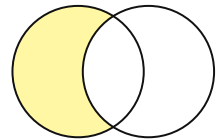
Enviarles el saludo de fin de año.



INTERSECT

¿Quién está en **las dos**?

Darles el descuento de personal.



EXCEPT

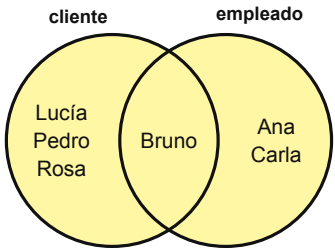
¿Quién es cliente y **no** empleado?

Enviarles la promoción de noviembre.

Combinar SELECT

Se escriben dos `SELECT` y, entre ellos, un operador. El resultado es **una sola lista**. `JOIN` agrega **columnas**; estos operadores trabajan con **filas**.

`UNION`: todo lo de las dos listas




```sql
SELECT dni, nombre FROM cliente
UNION
SELECT dni, nombre FROM empleado;
```

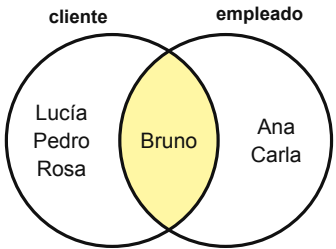
resultado

dni	nombre
45123678	Lucía Ramos
46234789	Pedro Salas
47345890	Rosa Medina
48456901	Bruno Díaz
43567012	Ana Torres
44678123	Carla Núñez

En palabras simples

Bruno Díaz está en las dos tablas con el mismo DNI y sale **una sola vez**: `UNION` quita los repetidos. Con `UNION ALL`, que los deja, saldría dos veces.

INTERSECT: solo lo que está en las dos




```sql
SELECT dni, nombre FROM cliente
INTERSECT
SELECT dni, nombre FROM empleado;
```

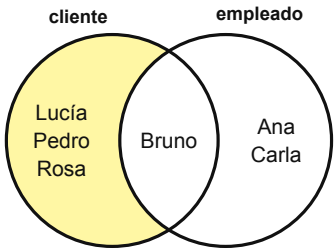
resultado

dni	nombre
48456901	Bruno Díaz

En palabras simples

Bruno Díaz está en **las dos listas** con el mismo DNI: le corresponde el descuento de personal. El DNI evita confundir a dos personas que se llaman igual.

EXCEPT: lo que está en la primera y no en la segunda




```sql
SELECT dni, nombre FROM cliente
EXCEPT
SELECT dni, nombre FROM empleado;
```

resultado

dni	nombre
45123678	Lucía Ramos
46234789	Pedro Salas
47345890	Rosa Medina

En palabras simples

Lucía, Pedro y Rosa son clientes y **no trabajan aquí**: reciben la promoción. **El orden importa**: al revés, empleado `EXCEPT` cliente da **Ana** y **Carla**.

Reglas para combinar dos `SELECT`

1. La misma **cantidad de columnas** en los dos `SELECT`.
2. Las columnas en el mismo **orden** y con tipos compatibles.
3. Los **nombres** de las columnas salen del primer `SELECT`.
4. Un solo `ORDER BY`, al **final** de todo.

```
SELECT nombre, telefono FROM cliente UNION SELECT nombre FROM empleado;
ERROR 1222 (21000): The used SELECT statements have a different number of
columns
```

Ojo con esto

`INTERSECT` y `EXCEPT` funcionan desde **MySQL 8.0.31**. Revisa tu versión con `SELECT VERSION();`. En versiones anteriores dan error de sintaxis y se reemplazan con `IN` y `NOT IN`.

EXCEPT en los dos gestores

Abre `ejemplos/practicas/practica_2_combinar.sql` y ejecútalo en los dos gestores.

1. **MySQL:** en Workbench.
2. **SQL Server:** en SSMS.

¿Qué servicios cuestan menos de S/ 60, sin contar los que duran una hora o más?

```
USE peluqueria;
SELECT nombre, precio FROM servicio
WHERE precio < 60
EXCEPT
SELECT nombre, precio FROM servicio
WHERE duracion >= 60
ORDER BY precio;
```

resultado

nombre	precio
Corte de uñas	15.00
Baño e higiene	35.00

Idea clave

La lista sale igual en los dos gestores: `EXCEPT` se escribe de la misma forma.

Subconsultas

Una pregunta que necesita otra pregunta

¿Qué servicios cuestan **más que el promedio**?

1 Primero, ¿cuál es el promedio?

```
SELECT AVG(precio) FROM servicio;
```

resultado

AVG(precio)
46.250000

2 Luego, ¿qué servicios cuestan más?

```
SELECT nombre, precio FROM servicio
WHERE precio > 46.25;
```

resultado

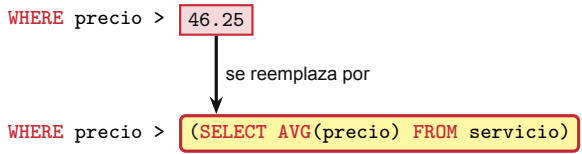
nombre	precio
Corte de pelo	55.00
Baño completo con corte	80.00

Ojo con esto

Funciona, pero copiaste `46.25` **a mano**. Si mañana cambia un precio, ese número queda viejo.

En una sola: la subconsulta

En lugar de escribir el número, escribe **la consulta que lo calcula**:




```sql
SELECT nombre, precio
FROM servicio
WHERE precio > (SELECT AVG(precio) FROM servicio);
```

Subconsulta

Una consulta escrita **dentro de otra**, entre **paréntesis**. El gestor resuelve **primero** la de adentro y usa su resultado en la de afuera.

¿Qué devuelve la de adentro?

Lo primero es mirar la **forma** de su resultado. Eso decide cómo se usa.

Un valor

46.250000

una fila, una columna

= > <

se compara

Una lista

1
2

una columna, varias filas

IN NOT IN

se busca dentro

Una tabla

1	4
2	3

varias columnas y filas

FROM

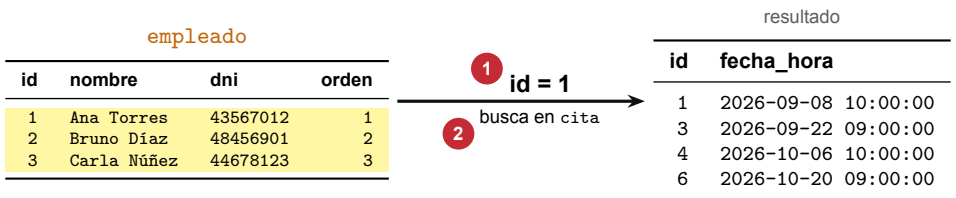
se usa como tabla

Ojo con esto

Si la de adentro devuelve una lista y usas =, sale `ERROR 1242: Subquery returns more than 1 row`. El signo = espera un solo valor; para una lista, usa `IN`.

En `WHERE`: un valor de otra tabla

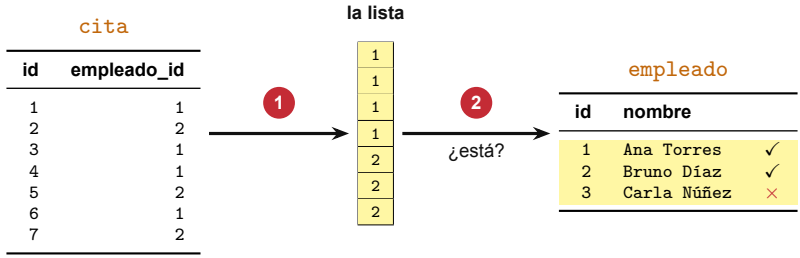
¿Qué citas tiene Ana Torres? En `cita` no está su nombre, solo su `empleado_id`. **No sabes** ese id: que lo busque la subconsulta.




```sql
SELECT id, fecha_hora FROM cita
WHERE empleado_id = (SELECT id FROM empleado WHERE nombre = 'Ana Torres');
```

Con `IN` y `NOT IN`: ¿qué empleados tienen citas?

`IN` ya lo conoces: `WHERE id IN (1, 2)`. Ahora la lista la escribe **una consulta**.




```sql
SELECT nombre FROM empleado
WHERE id IN
    (SELECT empleado_id FROM cita);
```

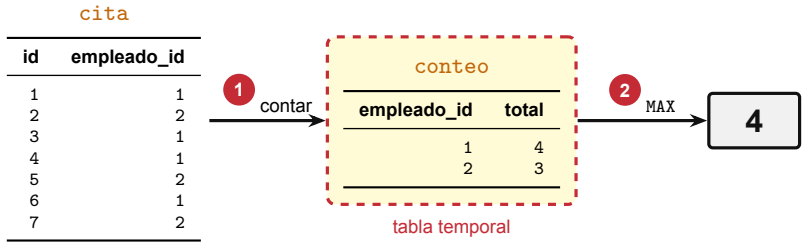
Ana Torres y **Bruno Díaz**

```sql
SELECT nombre FROM empleado
WHERE id NOT IN
    (SELECT empleado_id FROM cita);
```

Carla Núñez: aún no tiene citas

En FROM: ¿cuántas citas tiene el que más tiene?

`MAX(COUNT(*))` **no se puede** escribir. Primero cuentas; luego buscas el máximo.




```
SELECT MAX(total)
FROM (SELECT empleado_id, COUNT(*) AS total
      FROM cita GROUP BY empleado_id) AS conteo;
```

Ojo con esto

En `FROM`, la subconsulta **necesita un nombre** (`AS conteo`). Sin él: `ERROR 1248: Every derived table must have its own alias.`

En SELECT: una columna más

Si la de adentro devuelve **un valor**, puede ir como una columna. ¿Cuántas citas tiene **cada** empleado? Aquí la de adentro usa `e.id`, que viene de la consulta de afuera.

```
SELECT e.nombre,
    (SELECT COUNT(*) FROM cita c WHERE c.empleado_id = e.id) AS citas
FROM empleado e;
```

Fila de afuera	La de adentro cuenta…	citas
Ana Torres, `id = 1`	las citas con `empleado_id = 1`	**4**
Bruno Díaz, `id = 2`	las citas con `empleado_id = 2`	**3**
Carla Núñez, `id = 3`	las citas con `empleado_id = 3`	**0**

Subconsulta correlacionada

Usa una columna de afuera. Por eso **no** se resuelve una sola vez: se ejecuta **una vez por cada fila** de afuera.

EXISTS: ¿hay al menos una fila?

No necesitas un dato. Solo quieres saber si **existe**: ¿este empleado tiene alguna cita?

```
SELECT e.nombre
FROM empleado e
WHERE EXISTS (SELECT 1 FROM cita c WHERE c.empleado_id = e.id);
```

Fila de afuera	¿Tiene alguna cita?	¿Se queda?
Ana Torres, `id = 1`	sí: encontró la cita 1 y dejó de buscar	✓
Bruno Díaz, `id = 2`	sí: encontró la cita 2 y dejó de buscar	✓
Carla Núñez, `id = 3`	no: revisó todas y no hay	×

EXISTS

Responde **sí o no**: sí, cuando la de adentro encuentra **al menos una fila**. No mira los valores, por eso se suele escribir `SELECT 1`.

NOT EXISTS: que no haya ninguna

Es la pregunta contraria. Sirve para encontrar lo que **quedó sin relación**.

Empleados sin citas

```
SELECT e.nombre
FROM empleado e
WHERE NOT EXISTS (SELECT 1
   FROM cita c
   WHERE c.empleado_id = e.id);
```

resultado

nombre
Carla Núñez

Citas atendidas sin cobrar

```
SELECT c.id, c.fecha_hora
FROM cita c
WHERE c.estado = 'completada'
   AND NOT EXISTS (SELECT 1
      FROM pago p
      WHERE p.cita_id = c.id);
```

resultado

id	fecha_hora
6	2026-10-20 09:00:00

En palabras simples

La segunda también sale con `EXCEPT`: citas completadas menos citas en `pago`. Con `NOT EXISTS` puedes mostrar más columnas, como la fecha.

EXISTS vs IN

Las dos consultas dan el mismo resultado: **Ana Torres** y **Bruno Díaz**.

IN

```
SELECT nombre FROM empleado
WHERE id IN
    (SELECT empleado_id
    FROM cita);
```

EXISTS

```
SELECT e.nombre FROM empleado e
WHERE EXISTS
    (SELECT 1 FROM cita c
    WHERE c.empleado_id = e.id);
```

	IN	EXISTS
Pregunta	¿este valor está en la lista?	¿hay al menos una fila?
La de adentro	se puede ejecutar sola	usa una columna de afuera
Cómo trabaja	arma la lista y busca en ella	prueba fila por fila
Conviene si	comparas una columna con una lista	necesitas varias condiciones

En palabras simples

Hoy los gestores suelen ejecutar las dos igual de rápido: elige la que se lea mejor. La diferencia real aparece con `NOT IN`.

Ojo: `NOT IN` y los `NULL`

```sql
SELECT nombre FROM empleado
WHERE id NOT IN (1, 2);
```

resultado

nombre
Carla Núñez

```sql
SELECT nombre FROM empleado
WHERE id NOT IN (1, 2, NULL);
```

resultado

ninguna fila

¿Por qué? Para Carla, el gestor compara su id con cada valor de la lista:

`3 <> 1` ✓ `3 <> 2` ✓ `3 <> NULL` **no se sabe**

Ojo con esto

`NULL` es un valor desconocido. Si la lista tiene un `NULL`, el gestor no puede asegurar que el valor no esté, y descarta la fila. Pasa con columnas que admiten `NULL`, como `cliente.telefono`. `NOT EXISTS` no tiene este problema.

Subconsulta en los dos gestores

Abre `ejemplos/practicas/practica_3_subconsulta.sql` y ejecútalo en los dos gestores.

1. **MySQL:** en Workbench.
2. **SQL Server:** en SSMS.

```sql
USE peluqueria;
SELECT nombre, precio
FROM servicio
WHERE precio > (SELECT AVG(precio) FROM servicio);
```

resultado

nombre	precio
Corte de pelo	55.00
Baño completo con corte	80.00

Idea clave

La subconsulta también se escribe igual en los dos gestores.

Expresiones de tabla comunes

Cuando la consulta crece

¿Qué empleado tiene **más citas**? Con lo que sabemos, se puede escribir así:

```
SELECT e.nombre, conteo.total
FROM empleado e
JOIN (SELECT empleado_id, COUNT(*) AS total FROM cita GROUP BY empleado_id) AS conteo
   ON conteo.empleado_id = e.id
WHERE conteo.total = (SELECT MAX(total)
   FROM (SELECT empleado_id, COUNT(*) AS total FROM cita GROUP BY empleado_id) AS otro);
```

Funciona y devuelve **Ana Torres, 4**. Pero tiene dos problemas:

- El mismo conteo está escrito **dos veces**.
- Se lee de **adentro hacia afuera**: hay que saltar por todo el código para entenderla.

CTE: una subconsulta con nombre

CTE (*Common Table Expression*)

Una subconsulta a la que le pones **un nombre**, escrita **antes** de la consulta principal. En español: expresión de tabla común.

```
WITH conteo AS (
    SELECT empleado_id, COUNT(*) AS total
    FROM cita GROUP BY empleado_id
)
SELECT MAX(total) FROM conteo;
```

- le pones un nombre
- cuenta las citas…
- … de cada empleado
- se cierra, sin punto y coma
- la consulta la usa como tabla

Se escribe una vez y se usa dos

Volvamos a la pregunta larga: ¿qué empleado tiene más citas?

```
WITH conteo AS (SELECT empleado_id, COUNT(*) AS total
         FROM cita GROUP BY empleado_id)
SELECT e.nombre, conteo.total
FROM empleado e
JOIN conteo ON conteo.empleado_id = e.id
WHERE conteo.total = (SELECT MAX(total) FROM conteo);
```

resultado

nombre	total
Ana Torres	4

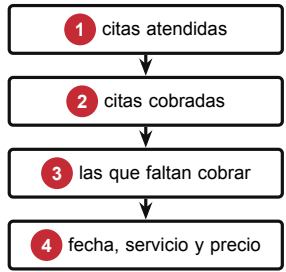
Idea clave

Si cambias cómo se cuenta, lo cambias en **un solo lugar**. Un CTE solo existe durante su consulta: **no** queda guardado en la base.

Todo junto: la lista de cobros

`WITH` se escribe **una vez** y los CTE se separan con **comas**. Cada uno puede usar los anteriores.

```
WITH atendidas AS (SELECT id FROM cita
            WHERE estado = 'completada'),
cobradas AS (SELECT cita_id FROM pago),
sin_cobrar AS (SELECT id FROM atendidas
            EXCEPT
            SELECT cita_id FROM cobradas)
SELECT c.id, c.fecha_hora,
   s.nombre AS servicio, s.precio
FROM sin_cobrar sc
JOIN cita c ON c.id = sc.id
JOIN servicio s ON s.id = c.servicio_id;
```



resultado

id	fecha_hora	servicio	precio
6	2026-10-20 09:00:00	Corte de pelo	55.00

Práctica 4 CTE en los dos gestores

Abre `ejemplos/practicas/practica_4_cte.sql` y ejecútalo en los dos gestores.

1. **MySQL:** en Workbench.
2. **SQL Server:** en SSMS.

```sql
USE peluqueria;
WITH caros AS
    (SELECT nombre, precio FROM servicio
    WHERE precio >
        (SELECT AVG(precio) FROM servicio))
SELECT COUNT(*) AS cantidad, SUM(precio) AS total
FROM caros;
```

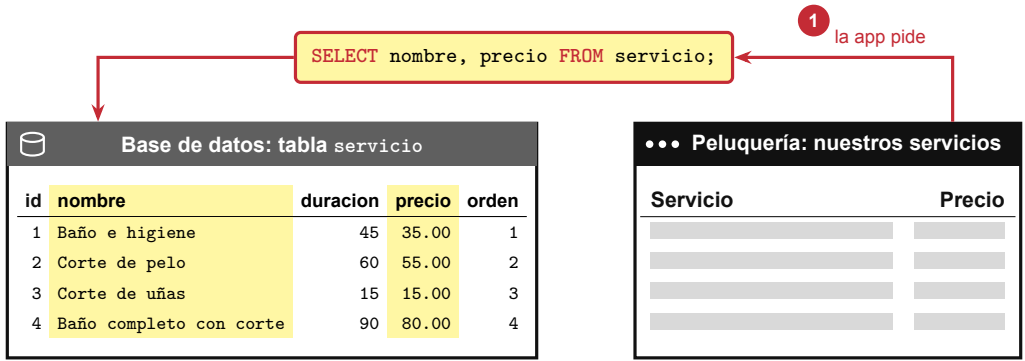
resultado

cantidad	total
2	135.00

Idea clave

Es la consulta de la práctica 3, ahora con nombre. El CTE también se escribe igual en los dos gestores.

Para esto sirven los `SELECT`: la app pide



La app pregunta

Abres la pantalla de servicios y todavía está vacía: la app le envía este `SELECT` a la base. Solo pide las dos columnas que va a mostrar: `nombre` y `precio`.

Para esto sirven los SELECT: la base responde

```
SELECT nombre, precio FROM servicio;
```

Base de datos: tabla `servicio`

id	nombre	duracion	precio	orden
1	Baño e higiene	45	35.00	1
2	Corte de pelo	60	55.00	2
3	Corte de uñas	15	15.00	3
4	Baño completo con corte	90	80.00	4

2 la base devuelve las filas

••• Peluquería: nuestros servicios

Servicio	Precio
Baño e higiene	S/ 35.00
Corte de pelo	S/ 55.00
Corte de uñas	S/ 15.00
Baño completo con corte	S/ 80.00

Ese es el uso de los SELECT

La tabla que ves en un aplicativo no se escribe a mano: sale de la base de datos con un `SELECT`. Si cambia un precio en la base, cambia en la pantalla.

Preguntas



Cierre

`UNION`	Junta las filas de dos `SELECT` sin repetir. `UNION ALL` deja los repetidos.
`INTERSECT`	Solo las filas que están en las dos listas.
`EXCEPT`	Las filas de la primera que no están en la segunda. El orden importa.
Subconsulta	Una consulta dentro de otra, entre paréntesis. Puede ir en `WHERE`, `FROM` o `SELECT`.
`IN`	Busca un valor en una lista. Cuidado con `NOT IN` y `NULL`.
`EXISTS`	Responde sí o no: ¿hay al menos una fila?
CTE	Una subconsulta con nombre, escrita antes con `WITH`. Solo vive durante su consulta.
En una app	Cada pantalla que muestra datos ejecuta un `SELECT`.

Referencias

- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2014). *Fundamentos de bases de datos* (6.ª ed.). McGraw-Hill.
- Oracle. MySQL Reference Manual: *UNION*, *INTERSECT* y *EXCEPT*.
- Oracle. MySQL Reference Manual: *Subqueries* y *Subqueries with EXISTS or NOT EXISTS*.
- Oracle. MySQL Reference Manual: *WITH (Common Table Expressions)*.



UTP Universidad Tecnológica del Perú